

Learning to Race from Rays: Reinforcement Learning for Autonomous Racing

University of Haifa

Reinforcement Learning, Spring 2026

Prof. Teddy Lazebnik

Adi Mishael - ID 206706442

Yarden Tziar - ID 208017749

Nissim Deri - ID 211569744

Baruh Riss - ID 318739174



אוניברסיטת חיפה
UNIVERSITY OF HAIFA
جامعة حيفا

1. PROJECT OVERVIEW

The goal of this project was to train a reinforcement learning agent to drive a Formula 1 car around racing circuits using only the information an on-board sensor system would provide. No map and prior knowledge of the track layout. The agent receives 30 distance readings from a LiDAR-like raycast sensor and four proprioceptive values about its own motion, and from that it must learn to navigate at high speed without crashing.

The motivation for this sensor-only framing comes from autonomous vehicles more broadly. A system that can handle new environments from local sensor data is much more useful than one that relies on a memorized map (e.g. Tesla's FSD vs Google's Waymo).

Racing is a useful environment because the task is well-defined, the failure modes are obvious, and the setting can be made arbitrarily complex by generating new tracks.

We built the simulation using Box2D as the physics engine and Pygame for rendering. The environment wraps the Gymnasium API, which let us plug in Stable-Baselines3's PPO implementation without writing a custom training loop. All track geometry, sensor logic, collision detection, reward computation, and multi-car dynamics were built by the team.

The main result from the completed phases is that a PPO agent trained on randomly generated tracks achieves 100% lap completion on five held-out tracks it never saw during training, across three independent seeds. When extended to multi-car racing with self-play opponents, the agent achieves 98% lap success while also learning to overtake.

2. PROBLEM FORMULATION

2.1 Environment

The simulation models a top-down 2D Formula 1 car using Box2D at 60 Hz.

The car is 5.6m long, 2.0m wide, and weighs 798 kg. Engine force is capped at 22 kN forward and 36 kN braking.

Lateral tire grip is modeled as a restoring impulse with a base coefficient of 0.92, increasing slightly with speed squared to simulate aerodynamic downforce. Steering authority decreases gradually at high speed to keep mid-speed cornering responsive.

Tracks are generated as closed Fourier-harmonic polar curves. Boundaries are derived from the centerline using Shapely's polygon buffer. Any generated track is rejected if the polygon is self-intersecting, if the inner boundary collapses to zero width, or if maximum curvature exceeds 0.2 m^{-1} (turn radius below 5m). In early development, this filter rejected more generated tracks than we expected. High-amplitude harmonic settings produced inner boundary collapse often enough that we had to tune the parameter ranges down before training was practical.

Two named evaluation tracks are built into the system. The Sprint Circuit is roughly 750m long with a 14m width and several tight corners. The Grand Prix Circuit is approximately 3.5km long with wider corners, a long straight, and a chicane section.

Frenet coordinates (arc-length progress s , lateral deviation e_y , heading error e_ψ) are computed at each step for reward computation. The agent cannot see them.

2.2 State Space

The observation is a 34-dimensional float32 vector. Indices 0-29 are normalized raycast distances in $[0,1]$ from 30 rays: 24 across a forward semicircle (-90 to $+90$ degrees) and 6 rear mirror rays. Indices 30-33 are proprioceptive values: speed/95, lateral velocity/50, previous throttle, and previous steering. Because a single frame carries no velocity information, the environment is wrapped with VecFrameStack($n_stack=4$), giving a 136-dimensional policy input from which the network infers motion by computing ray-distance deltas across frames [Mnih et al., 2015]. We considered using a recurrent

policy instead, but the gradient instability of backpropagation through time was not worth the added complexity for this task.

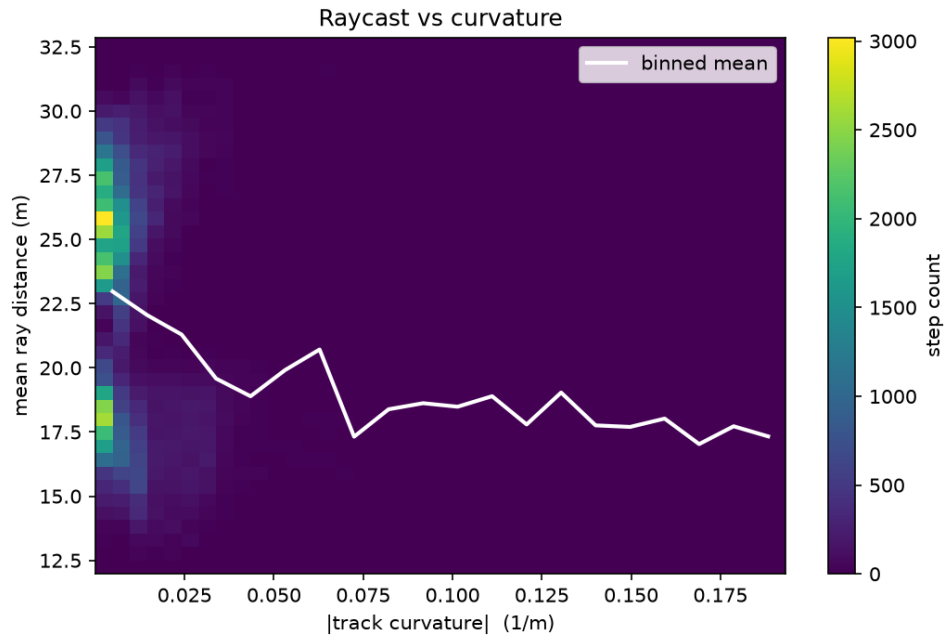


Figure 1: Raycast sensor coverage - mean measured distance per ray angle aggregated over a held-out lap, illustrating the 34-dimensional observation.

2.3 Action Space

The action space is a 2-dimensional continuous box. Index 0 is throttle in the range $[-1, 1]$: positive values apply engine force, negative values apply braking. Index 1 is steering in the range $[-1, 1]$ and maps directly to yaw torque.

2.4 Reward Function

The reward function was developed iteratively over Phases 2 and 3. The final single-car profile (v2) is:

<code>r = +1.0 * delta_s</code>	(forward centerline
<code>progress, meters)</code>	
<code>+ 0.01 * max(v_fwd, 0)</code>	(speed bonus, forward only)
<code>- 0.1 * ey / half_width</code>	(lateral deviation penalty)
<code>- 0.05 * e_psi / pi</code>	(heading error penalty)
<code>- 0.5 * delta_steering </code>	(control smoothness)
<code>- 10.0 * [new wall contact]</code>	(per-hit wall penalty)
<code>- 0.25 * [currently touching wall]</code>	(sustained wall contact)
<code>- 0.001</code>	(per-step time cost)

- 50.0 * [off-track] + termination
- 25.0 * [reverse termination]

During any step where the car is touching a wall, the `delta_s` progress reward and the speed bonus are both zeroed, making wall contact cost-only. The multi-car extension (v3) adds a +2.0 overtake reward, a -5.0 per-collision penalty, and a -0.15 per-contact-step penalty. The design rationale is discussed in Section 3.2.

2.5 Episode Termination

An episode terminates early if the car leaves the track boundaries (off-track, -50 penalty), remains stuck against a wall for 30 consecutive steps without forward progress, or makes backward arc-length progress for 120 consecutive steps (reverse termination, -25 penalty). If none apply, the episode truncates at 6000 steps (100 seconds at 60Hz).

3. METHODOLOGY

3.1 Algorithm: Proximal Policy Optimization

We used Proximal Policy Optimization (PPO) [Schulman et al., 2017] for all training. The action space is continuous and 2-dimensional, which rules out discrete value-based methods. We also considered SAC (Soft Actor-Critic [Haarnoja et al., 2018]), which is generally more sample-efficient on continuous control tasks. The reason we used PPO is data generation rate. Our simulator runs at 60Hz with 8 parallel environments, generating approximately 480 physics steps per second. PPO consumes on-policy rollouts immediately, which matches this rate well. Off-policy replay-buffer methods add less value when data generation is cheap and continuous.

The policy is a multi-layer perceptron with two shared hidden layers of 256 units (tanh activations), with separate actor and critic heads.

Table 1: PPO hyperparameters used across all phases.

Parameter	Value	Rationale
net_arch	[256,256]	Sufficient for 136-dim stacked input
n_steps	2048	~34 sec of rollout per env per update
batch_size	64	Standard for on-policy updates
n_epochs	10	Each rollout is reused 10 times
gamma	0.99	High discount for long episodes (6000 steps)
gae_lambda	0.95	Biased toward lower variance over MC
clip_range	0.2	From PPO paper recommendation
ent_coef	0.01	Small entropy bonus for exploration
learning_rate	3e-4	Adam optimizer, standard for SB3 continuous

Advantage estimates use GAE [Schulman et al., 2016] with $\lambda=0.95$, which biases toward lower variance while retaining most of the benefit of longer returns. The effect of this choice is studied empirically (see Section 3.6).

Model selection uses SB3's EvalCallback, evaluating the current policy every 250,000 steps on a fixed validation track (procedural seed 2001) for 3 episodes. The best checkpoint by mean validation reward is saved separately. Seed 2001 is never used for training and is not part of the held-out test set, keeping training, validation, and test strictly separate.

3.2 Reward Engineering: From v1 to v2

The first training run used reward profile v1 on the Sprint Circuit for 1 million timesteps. The learning curve rose consistently, which initially looked like the agent was learning to drive. When we watched the trained agent, it was driving backwards.

The v1 reward uses a signed Δs term for centerline progress. Crossing the start/finish line in reverse registers as positive Δs , just like forward motion. By reversing in circles near the start line, the agent accumulated continuously increasing reward without completing a lap. All three seeds under v1 reached 0% lap completion with a mean progress fraction of -8.01.

This is the failure pattern Ng et al. [1999] describe: the agent optimized the reward as specified, not the intended behavior. The fix required restructuring the reward so that reversing was no longer profitable.

The v2 profile makes four changes. The speed bonus uses $\max(v_{\text{fwd}}, 0)$ so reversing earns no speed reward. Both progress reward and speed bonus are zeroed during wall contact, which also closes the wall-scraping exploit observed in early v1 training. A per-step time cost of -0.001 discourages idling. Finally, a reverse-termination condition ends episodes after 120 consecutive backward steps with a -25 penalty.

3.3 Training Protocol and Domain Randomization

Phases 2 and 3 trained exclusively on the Sprint Circuit. To prevent the agent from memorizing a single track's geometry, Phase 4 uses domain randomization [Tobin et al., 2017]: a new procedural track is generated from a fresh random seed at each episode reset.

The training distribution samples base radius from [150, 500]m and track width from [18, 28]m. Procedural seeds 1001, 1002, 1003, and validation seed 2001 are excluded from the generator. Training used 8 parallel environments for 5 million timesteps, for seeds 42, 43, and 44.

The held-out evaluation set is: the Sprint Circuit, the Grand Prix Circuit, and procedural seeds 1001, 1002, and 1003. Each track was evaluated for 20 episodes per seed (300 total), with randomized starting positions and small lateral and heading perturbations.

3.4 Multi-Car Curriculum

After Phase 4, the agent could drive arbitrary procedural tracks in isolation. Adding opponents abruptly creates a problem: constant car-to-car collisions overwhelm the driving reward before the agent has any collision-avoidance behavior. The approach is a staged curriculum where each stage adds difficulty gradually and warm-starts from the previous stage's best checkpoint [Bengio et al., 2009].

Stage 5b adds one stationary car at a random arc-length position. This was expected to be easy, and it was: a parked car produces the same ray distance compression as a wall stub, so the Phase 4 policy transferred with almost no adaptation.

Stage 5c introduces one car driving the centerline at 50% speed using kinematic control. This was the first stage requiring genuinely new behavior. The validation reward showed a clear spike around 1 million steps where sustained overtaking appeared to emerge. Initially we expected this behavior to develop gradually, but the validation curve was flat for most of Stage 5c before the shift appeared in a fairly short window.

Stage 5d adds three centerline-following cars at 50% speed. Validation reward was more variable: the agent sometimes threaded gaps cleanly and sometimes got stuck behind a cluster. Stage 6 replaces scripted opponents with learned policies from a checkpoint pool (Section 3.5).

The v3 multi-car reward extends v2 with a -5.0 per-collision penalty, a -0.15 per-contact-step penalty, and a +2.0 overtake reward for each opponent passed.

3.5 Self-Play with a Checkpoint Pool

Scripted opponents have fixed, predictable behavior. Once the agent memorizes their trajectory pattern, the stage stops challenging general racing skill. A more principled approach is to train against opponents following their own learned policies.

Training against a current copy of the policy creates a non-stationary environment: as the agent improves, so do the opponents, causing instabilities and cycling [Silver et al., 2018]. The checkpoint pool approach trains the agent against a pool of past policy versions instead.

A callback saves a snapshot to a pool directory every 200,000 steps. The pool is capped at 10 snapshots; the oldest is evicted when it fills. At each episode reset, each of the 3 opponents draws a random snapshot from the current pool. We had not included the cap in early runs and found the agent spending most episodes against the earliest incompetent snapshots, which provided almost no useful training signal. Pool

opponents use full Box2D physics with their own observation buffers and raycast sensors, producing physically realistic behavior that scripted bots cannot replicate.

Stage 6 was started from the Stage 5d best model and trained for 1.5 million steps.

3.6 Advanced Training Techniques (Phase 7)

3.6.1 GAE Lambda Ablation

The `gae_lambda` hyperparameter controls the bias-variance tradeoff in advantage estimation. At $\lambda=0$, advantages are one-step TD errors: low variance but dependent on value function accuracy. At $\lambda=1$, advantages are full Monte Carlo returns: unbiased but high variance. We sweep λ over $\{0.0, 0.5, 0.9, 0.95, 1.0\}$ with 3 seeds each under the Phase 4 setup. The 0.95 value was inherited from the PPO paper defaults. We had no strong reason to believe it was well-matched to this specific combination of episode length and track variety, which is what motivated the Phase 7 sweep.

The held-out results for this ablation are reported in Section 4.5 (Figures 4 and 5).

3.6.2 Auxiliary Raycast Prediction

The policy trunk is extended with a second output head that predicts the next 30-ray observation given the current observation and action, updated via a separate backward pass at the end of each PPO rollout, with MSE loss at coefficient 0.05. A representation that predicts future sensor states must encode local environment geometry, which should produce more transferable features across track shapes. This is similar in spirit to the auxiliary tasks in UNREAL [Jaderberg et al., 2017] and the general value function framework [Sutton et al., 2011].

The auxiliary head was implemented; its Sigmoid output activation architecturally constrains predictions to $[0,1]$, matching the normalised ray-distance range. A full head-to-head comparison against the baseline policy on the held-out set was not completed

within the project timeline. We therefore report it as an implemented technique rather than a confirmed empirical result

3.6.3 Adaptive Reward Normalization

Reward magnitudes shift across curriculum stages as the v3 multi-car terms are added. SB3's VecNormalize wrapper normalizes rewards online via a running mean and variance, keeping critic value targets on a stable scale regardless of which reward profile is active.

Reward normalization was enabled for the GAE-lambda sweep reported in Section 4.5 rather than ablated in isolation, so its effect is reported jointly with that sweep rather than as a standalone comparison.

4. RESULTS

4.1 Reward Shaping: v1 vs. v2 on Sprint Circuit

Table 2: Phase 3 results on the Sprint Circuit. Mean and standard deviation across 3 seeds (42, 43, 44). Each seed evaluated on 5 episodes.

Metric	v1 (control)	v2 (shaped)
Lap completion rate	0% +/- 0%	100% +/- 0%
Laps per episode	0 +/- 0	8 +/- 0
Progress fraction	-8.01 +/- 0.14	+8.49 +/- 0.07
Wall hits per episode	29.4 +/- 1.7	0.5 +/- 0.7

The v1 agent discovered the reverse-driving exploit and followed it consistently across all three seeds. The learning curve rose over training, so there was no indication from the reward signal alone that anything was wrong. The behavioral evaluation was the only way to catch this.

After the v2 fixes, all three seeds achieved 100% lap completion with near-zero wall hits. The progress fraction flipped from -8 to +8.5, meaning the agent covered roughly 8.5 track lengths per episode forward instead of backward. Raw episode returns between v1 and v2 are not comparable because the reward scales differ; lap completion and progress fraction are the meaningful metrics.

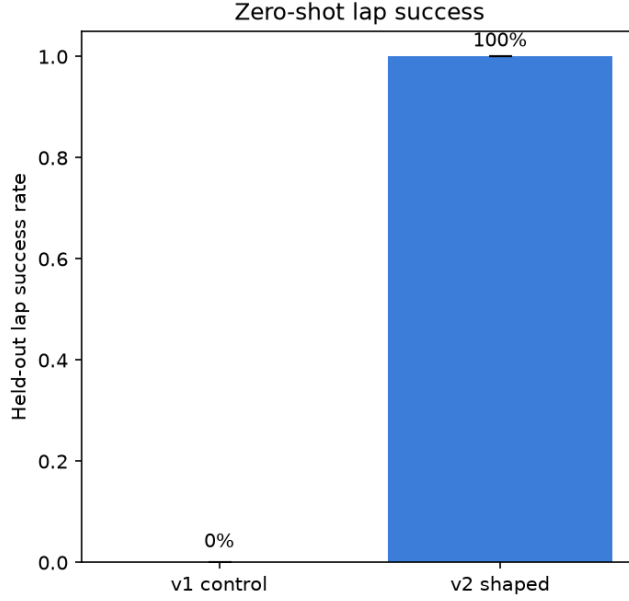


Figure 2: Zero-shot held-out lap success for the v1 control reward (0%, reverse-driving exploit) versus the v2 shaped reward (100%), mean over three seeds.

4.2 Domain Randomization: Held-Out Evaluation

Table 3: Phase 4 held-out evaluation results. Mean wall hits and speed averaged across 3 seeds. All tracks: 20 episodes per seed = 60 episodes per track total.

Track	Lap success	Wall hits (mean)	Speed (m/s)
Sprint Circuit	100%	38.0	57.1
Grand Prix	100%	5.1	77.1
Procedural 1001	100%	0.4	88.6
Procedural 1002	100%	4.0	86.5
Procedural 1003	100%	10.3	82.6
Aggregate mean	100% +/- 0%	11.6 +/- 4.9	78.4 +/- 0.2

Every track reached 100% lap completion across all 300 evaluation episodes. The three procedural held-out tracks show the clearest generalization: entirely different Fourier coefficients from any training track, handled with relatively few wall contacts.

The Sprint Circuit is the outlier. Its 14m width falls below the 18 to 28m training range, so the agent enters tight corners slightly too fast. It still completes every lap, but with more wall contact. Sprint wall hits varied substantially across seeds (51.0, 8.6, 54.5), all at 100% lap success. Interestingly, seed 43 produced a policy with nearly six times fewer Sprint wall hits than the other two seeds, despite all three reaching 100%

success. We do not have a clear explanation for this; all three seeds used the same configuration and the difference appears to be stochastic variation in training.

The speed pattern follows track geometry: Sprint forces slower speeds (57.1m/s) while the wide procedural tracks allow 83-89 m/s.

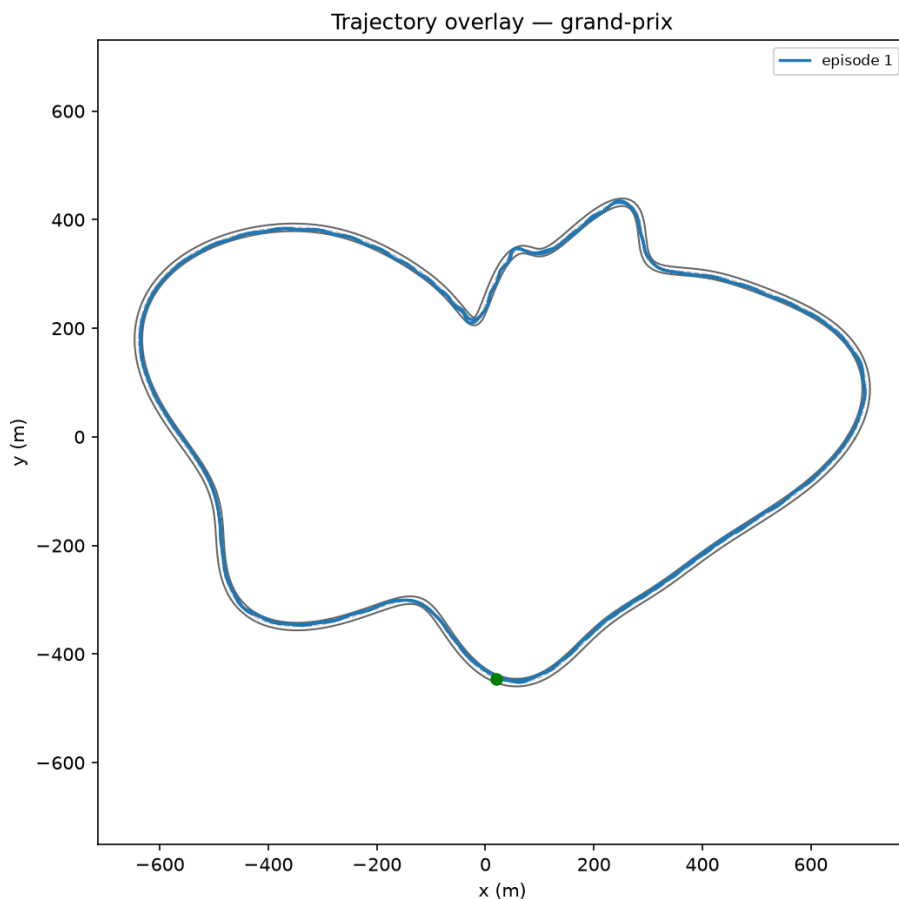


Figure 3: A trained agent's driven trajectory on the held-out Grand Prix circuit (zero-shot generalization to an unseen layout).

4.3 Multi-Car Curriculum Progression

The curriculum was run for seeds 42, 43, and 44. Best validation rewards for seed 42 (on validation seed 2001) at each stage were:

Stage 5b (1 stationary opponent): 11,147 (200k step stage)

Stage 5c (1 slow centerline follower): 11,765 (750k step stage)

Stage 5d (3 slow centerline followers): 11,322 (500k step stage)

Stage 6 (self-play pool): 11,217 at step 1.25M of 1.5M stage budget

Stage 5b was solved with little visible change in training dynamics. Stage 5c showed a clear shift around 1 million steps corresponding to the emergence of consistent overtaking. Stage 5d was harder and more variable, with validation rewards depending on whether the agent threaded gaps cleanly or got stuck behind a cluster. Stage 6 showed the high variance typical of self-play, with validation rewards ranging roughly between 5,000 and 11,000.

4.4 Phase 6: Self-Play Held-Out Evaluation

Table 4: Phase 6 held-out evaluation. Seed 42, 20 episodes per track, with 3 pool opponents active during each episode. Pool drawn from checkpoints saved between steps 1.2M and 3M.

Track	Success	Wall hits	Car collisions	Overtakes	Speed
Sprint Circuit	95%	26.9	7.0	5.8	49.1
Grand Prix	95%	8.1	1.4	0.6	75.1
Procedural 1001	100%	0.6	0.4	0.3	86.3
Procedural 1002	100%	2.2	0.7	0.7	82.9
Procedural 1003	100%	5.3	1.4	1.9	79.7
Mean	98%	8.6	2.2	1.9	74.6

The overall lap success rate dropped from 100% in Phase 4 (mean across seeds 42–44) to 98% in Phase 5e (seed 42). This is expected: pool opponents can collide with the ego agent and block narrow sections. Maintaining 98% across 5 tracks, including 2 outside the training width distribution, indicates the multi-car policy retained most of its generalization capability.

Sprint Circuit wall hits dropped from 38.0 (Phase 4 three-seed mean) to 26.9 (Phase 6, seed 42), despite added traffic. The most likely explanation is that navigating around cars on a narrow track forces more precise lateral positioning, which incidentally reduces wall contact. The two 5% failure rates on Sprint and Grand Prix both involved reverse-termination: after a collision, the agent sometimes entered a backward spin that triggered the 120-step cutoff.⁶ (with opponents), despite added traffic. The most likely explanation is that navigating around cars on a narrow track forces more precise lateral positioning, which incidentally reduces wall contact. The two 5% failure rates on Sprint

and Grand Prix both involved reverse-termination: after a collision, the agent sometimes entered a backward spin that triggered the 120-step cutoff.

Overtakes averaged 1.9 per episode. The behavior emerged from the +2.0 overtake reward and the agent's forward progress incentive; no additional engineering was needed for it to appear.

4.5 Phase 7 Results

Figure 4 shows the held-out evaluation return over training for five values of the GAE parameter λ , and Figure 5 the corresponding zero-shot lap-success rates. The result is a clean bias-variance picture. With $\lambda = 0.0$ and 0.5 the advantage estimator is heavily biased toward short-horizon returns; learning is unstable and reaches only 34-37% held-out lap success with very high seed-to-seed variance. From $\lambda = 0.9$ upward the estimator captures enough of the long-horizon credit needed for cornering and the agent learns reliably, reaching roughly 100% success. $\lambda = 0.95$ gives the best overall balance - near-perfect success with the lowest wall-contact rate and a strong, low-variance return curve - and is the value used for every other phase in this project. $\lambda = 1.0$ (full Monte-Carlo returns) attains the highest raw return and fastest laps but with noisier learning and more wall contact. Reward normalization (Section 3.6.3) was enabled for all runs in this sweep, so the comparison isolates λ within the normalized-reward setting.

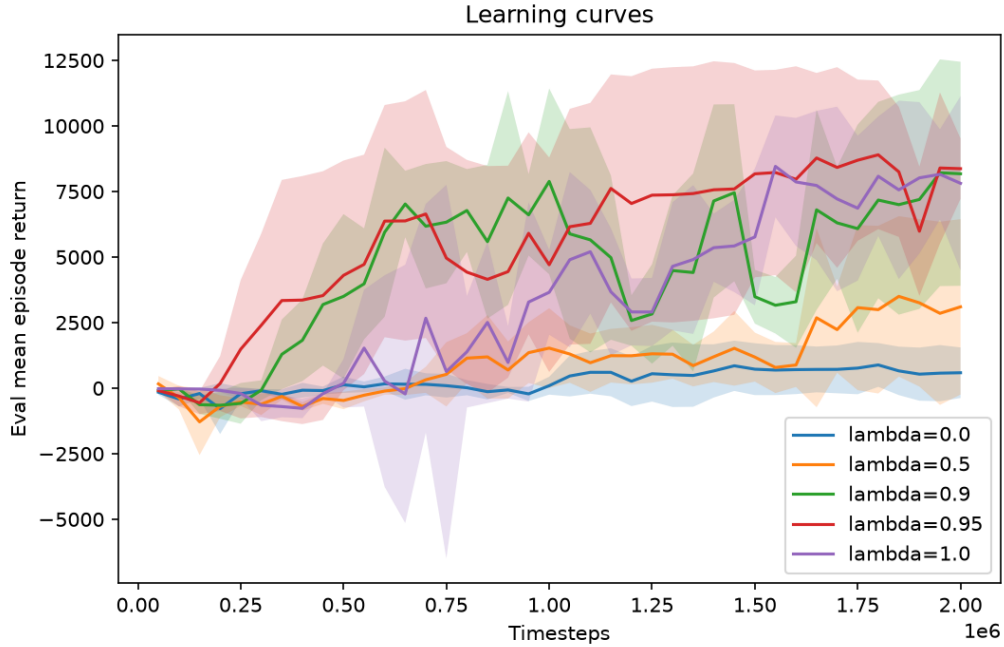


Figure 4: GAE-lambda ablation - held-out evaluation return over training (shaded band = ± 1 std across seeds).

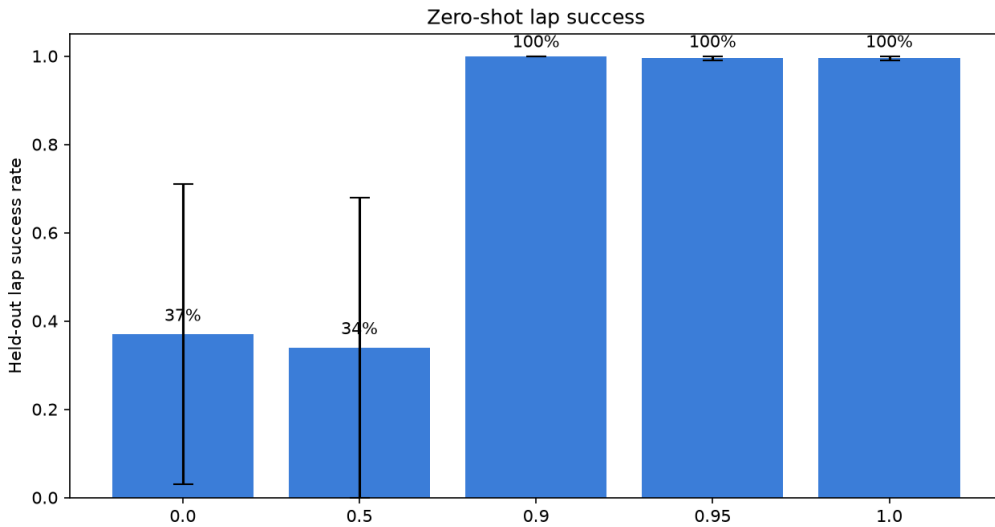


Figure 5: GAE-lambda ablation - held-out zero-shot lap-success rate by lambda (error bars = ± 1 std across seeds).

5. DISCUSSION

5.1 Reward Engineering Lessons

The v1 reverse-driving failure is a good example of a pattern that appears frequently in RL. The reward function said "increase forward progress along the centerline." The

agent found that backward progress through the start/finish line also satisfies this, and that doing so repeatedly is stable and profitable. The agent was optimizing the reward exactly as specified.

Our first instinct was to add a penalty: "if the agent is reversing, penalize it." This does not work well if the positive reward for the exploit exceeds the penalty. The agent will still reverse if the net reward is positive, and will simply trade off reversing time against penalty time. The correct approach is to redesign the reward so that the exploit is no longer profitable, not to try to penalize it away. In our case, the fix was to make backward progress give zero positive reward (by using $\max(v_fwd, 0)$ and eventually the reverse termination), so there was no longer a reason for the agent to reverse in the first place.

The same logic applies to wall scraping. Adding a large wall-hit penalty might reduce the frequency of hits, but if sliding along the wall is still the fastest path through a corner, the agent will keep doing it until the penalty exceeds the speed advantage. Zeroing the progress and speed rewards during wall contact changed the question from "is the wall penalty large enough" to "is there any reason to touch the wall at all."

5.2 What Domain Randomization Actually Generalizes

Domain randomization generalizes well within the Fourier-harmonic track distribution. Held-out procedural seeds 1001-1003 were generated by the same process with different parameters, and all achieved 100% success with low wall hits.

Sprint represents out-of-distribution generalization on a single parameter: track width. Everything else about Sprint (turn radii, overall shape) is within the training range, but its 14m width is 4m below the training minimum. The agent handles it successfully but with more wall contact, which is the expected result of being calibrated to wider tracks.

Extending the lower bound of the width distribution to 14m or below would likely close most of the Sprint wall-hit gap with no other changes.

5.3 Self-Play and Emergent Behavior

Overtaking in Stage 6 was not scripted or directly shaped beyond the +2.0 overtake reward. It emerged from training. Opponents appear as ray compressions ahead of the car, and the agent learned that moving around a slower object generates more cumulative reward than following it. No additional engineering was needed for this behavior to appear.

The self-play dynamics also showed that the checkpoint pool works as intended. In early Stage 5e, the pool contained mostly Phase 5d-level agents that drove the centerline predictably. Over time, as the ego agent improved and added more capable snapshots, the pool shifted toward harder opponents. The validation reward variance reflects this shifting difficulty: high variance is typical when the opponent distribution changes faster than the policy can adapt.

We expected Sprint wall hits to increase in Phase 6 relative to Phase 4, since opponents add traffic to an already narrow circuit. Instead they dropped from 38.0 to 26.9. The current explanation is that navigating around pool opponents

on the narrow circuit requires more lateral precision than driving it empty, and that precision transfers into fewer wall contacts. This would be worth testing by evaluating the Phase 6 policy on Sprint without opponents, to check whether the wall hit reduction is due to opponents being present during evaluation or due to improved policy quality from the multi-car training.

5.4 Limitations

The clearest limitation is the Sprint Circuit width gap. Extending the training width distribution down to 12m would likely reduce Sprint wall hits with no other changes.

The multi-car curriculum was completed across three seeds (42, 43, and 44), matching the single-car evaluation protocol. Seed-to-seed variance in the multi-car setting is higher than in Phase 4, reflecting the added non-stationarity of self-play training.

The agent cannot anticipate corners before the forward rays reach them. At high speed, the gap between when a corner first appears in the raycast pattern and when braking needs to start can be smaller than the agent's effective reaction time. After looking at

recorded failure episodes, the pattern was consistent: the car typically had enough time in the forward raycast field to start braking, but did not reduce speed until the rays were already quite short. Adding longer- range forward rays or a Frenet-based curvature lookahead could address this, though it would partially break the sensors-only premise of the design.

6. CONCLUSION

This project trained a PPO agent for autonomous racing from raycast sensor data only, building all simulation infrastructure from scratch. Three key results stand out from the completed phases. Reward engineering required structural redesign rather than penalty patching to close the reverse-driving exploit. Domain randomization achieved 100% zero-shot lap success on five unseen tracks across three independent seeds. The multi-car self-play curriculum maintained 98% lap success while producing emergent overtaking behavior without any scripted assistance.

REFERENCES

- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. (2009). Curriculum learning. Proceedings of the 26th International Conference on Machine Learning (ICML), pp. 41-48.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 1861-1870.
- Jaderberg, M., Mnih, V., Czarnecki, W. M., Schaul, T., Leibo, J. Z., Silver, D., and Kavukcuoglu, K. (2017). Reinforcement learning with unsupervised auxiliary tasks. Proceedings of the 5th International Conference on Learning Representations (ICLR).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and

- Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533.
- Ng, A. Y., Harada, D., and Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278-287.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268), 1-8.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2016). High-dimensional continuous control using generalized advantage estimation. *Proceedings of the 4th International Conference on Learning Representations (ICLR)*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140-1144.
- Sutton, R. S., Modayil, J., Delp, M., Degris, T., Pilarski, P. M., White, A., and Precup, D. (2011). Horde: A scalable real-time architecture for learning knowledge from unsupervised sensorimotor interaction. *Proceedings of the 10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 761-768.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23-30.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulao, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Schulhoff, S., Tai, J., Tan, H.,

and Younis, O. M. G. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint [arXiv:2407.17032](https://arxiv.org/abs/2407.17032).